

Competitive Programming

Section 54: Geometry

Competitive Programming Course

2026-07-06

Table of contents

1	Geometry without floating point	3
1.1	Vectors and cross product	3
1.2	Segment intersection	3
1.3	Polygon double area	4
1.4	Geometry debugging checklist	4
1.5	Orientation worked example	4
1.6	Segment-intersection edge cases	5
1.7	C++ geometry primitive	5
2	Exercises	6

1 Geometry without floating point

Prefer integer cross products whenever coordinates are integral. Orientation, segment intersection, and polygon area all follow from the signed cross product. Delay floating point until output explicitly asks for it.

1.1 Vectors and cross product

For points $A(x_1, y_1)$, $B(x_2, y_2)$, $C(x_3, y_3)$, define:

```
AB = (x2-x1, y2-y1)
AC = (x3-x1, y3-y1)
cross(AB, AC) = AB.x * AC.y - AB.y * AC.x
```

Interpretation:

- positive: C is to the left of directed line $A \rightarrow B$;
- negative: C is to the right;
- zero: the three points are collinear.

Use 64-bit integers for coordinates up to about $1e9$, because products reach $1e18$.

1.2 Segment intersection

Two closed segments AB and CD intersect if:

1. Their orientations straddle each other:

```
orient(A,B,C) and orient(A,B,D) have opposite signs
orient(C,D,A) and orient(C,D,B) have opposite signs
```

2. Or one of the orientation values is zero and the collinear point lies inside the bounding box of the segment.

The collinear check:

```
min(A.x,B.x) <= P.x <= max(A.x,B.x)
min(A.y,B.y) <= P.y <= max(A.y,B.y)
```

This handles touching endpoints and overlapping collinear segments.

1.3 Polygon double area

For polygon vertices in order:

```
double_area = abs(sum cross(p[i], p[i+1]))
```

This is the shoelace formula. It returns twice the geometric area, avoiding fractions when coordinates are integral.

Diagram:

```
p0 ---- p1
|       /
|      /
p3--p2
```

The signed sum adds oriented trapezoids; taking absolute value removes clockwise/counterclockwise dependence.

1.4 Geometry debugging checklist

- Are segments closed or open?
- Do endpoints count as intersection?
- Are points guaranteed in polygon order?
- Can coordinates overflow 32-bit multiplication?
- Is exact integer output possible?

1.5 Orientation worked example

Let:

```
A = (0,0)
B = (4,0)
C = (2,3)
D = (2,-1)
```

For C:

```
AB = (4,0)
AC = (2,3)
cross = 4*3 - 0*2 = 12 > 0
```

So C is left of directed segment A → B.

For D:

```
AD = (2,-1)
cross = 4*(-1) - 0*2 = -4 < 0
```

So D is right of A $\rightarrow$ B.

1.6 Segment-intersection edge cases

Always test:

```
proper crossing
touching at one endpoint
parallel disjoint
collinear overlapping
collinear disjoint
same single-point segment
```

Most wrong geometry submissions fail one of the collinear cases, not the ordinary crossing case.

1.7 C++ geometry primitive

```
struct P {
    long long x, y;
};

long long cross(P a, P b, P c) {
    long long x1 = b.x - a.x;
    long long y1 = b.y - a.y;
    long long x2 = c.x - a.x;
    long long y2 = c.y - a.y;
    return x1 * y2 - y1 * x2;
}
```

Use this same primitive for orientation, segment intersection, and polygon area. Reusing one primitive reduces sign mistakes.

2 Exercises

- A. Orientation: Classify point C relative to directed segment AB.
- B. Segment Intersection: Decide whether two closed segments intersect.
- C. Polygon Double Area: Print twice the area of a polygon in vertex order. # Contest checklist

State the invariant, choose the model, prove the operation, then code the smallest reliable version.